In [ ]:
```
%load_ext autoreload
%autoreload 2

import sys
sys.path.append('../')
from scripts.data_utils import build_metadata

# This will scan your 30GB folder
df = build_metadata(raw_data_path='../data/raw')
df.head()
```

```
Scanning ../data/raw for paintings...
✅ Success! Mapped 81446 images to ../data/metadata.csv.
```

Out[ ]:

	path	style	artist
0	../data/raw/wikiart/Early_Renaissance/filippo-...	Early_Renaissance	filippo-lippi
1	../data/raw/wikiart/Early_Renaissance/filippo-...	Early_Renaissance	filippo-lippi
2	../data/raw/wikiart/Early_Renaissance/piero-de...	Early_Renaissance	piero-della-francesca
3	../data/raw/wikiart/Early_Renaissance/pietro-p...	Early_Renaissance	pietro-perugino
4	../data/raw/wikiart/Early_Renaissance/antonell...	Early_Renaissance	antonello-da-messina

In [ ]:
```
import torch
import torchvision.transforms as T
from torch.utils.data import Dataset, DataLoader, random_split
from sklearn.preprocessing import LabelEncoder
from PIL import Image
import random
import ssl


ssl._create_default_https_context = ssl._create_unverified_context

Image.MAX_IMAGE_PIXELS = None

device = torch.device("mps" if torch.backends.mps.is_available() else "cp
print(f"🚀 Using Device: {device}")


style_enc = LabelEncoder().fit(df['style'])
artist_enc = LabelEncoder().fit(df['artist'])

class ArtDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df = dataframe
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        while True:
            try:
```

```
                row = self.df.iloc[idx]
                image = Image.open(row['path']).convert('RGB')
                image.load()

                if self.transform:
                    image = self.transform(image)

                style_idx = style_enc.transform([row['style']])[0]
                artist_idx = artist_enc.transform([row['artist']])[0]
                return image, torch.tensor(style_idx, dtype=torch.long),

            except Exception:

                idx = random.randint(0, len(self.df) - 1)


transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

full_dataset = ArtDataset(df, transform=transform)


train_size = int(0.8 * len(full_dataset))
test_size = len(full_dataset) - train_size
train_data, test_data = random_split(full_dataset, [train_size, test_size

train_loader = DataLoader(train_data, batch_size=32, shuffle=True, num_wo
print(f"✅ Data Loaded! Training set: {len(train_data)} images.")
```

```
🚀 Using Device: mps
✅ Data Loaded! Training set: 65156 images.
```

In [ ]:

```
import ssl
ssl._create_default_https_context = ssl._create_unverified_context


from scripts.model import ArtCRNN
import torch.optim as optim
import torch.nn as nn

num_styles = len(style_enc.classes_)
num_artists = len(artist_enc.classes_)

print("Downloading ResNet18 Brain and Initializing Model...")
model = ArtCRNN(num_styles, num_artists).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 3

print("Starting Training Phase...")
for epoch in range(epochs):
    model.train()
    total_loss = 0

    for batch_idx, (images, style_targets, artist_targets) in enumerate(t
```

```
        images, style_targets, artist_targets = images.to(device), style_

        optimizer.zero_grad()
        style_preds, artist_preds = model(images)

        loss = criterion(style_preds, style_targets) + criterion(artist_p
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        if batch_idx % 50 == 0:
            print(f"Epoch [{epoch+1}/{epochs}] | Batch {batch_idx} | Curr

    print(f"✅ Epoch {epoch+1} Completed | Average Loss: {total_loss/len(
```

```
Downloading ResNet18 Brain and Initializing Model...
Starting Training Phase...
Epoch [1/3] | Batch 0 | Current Loss: 10.3640
Epoch [1/3] | Batch 50 | Current Loss: 9.1326
Epoch [1/3] | Batch 100 | Current Loss: 8.9564
Epoch [1/3] | Batch 150 | Current Loss: 8.4688
Epoch [1/3] | Batch 200 | Current Loss: 8.6763
Epoch [1/3] | Batch 250 | Current Loss: 8.2913
Epoch [1/3] | Batch 300 | Current Loss: 8.3974
Epoch [1/3] | Batch 350 | Current Loss: 9.2051
Epoch [1/3] | Batch 400 | Current Loss: 7.8183
Epoch [1/3] | Batch 450 | Current Loss: 8.2852
Epoch [1/3] | Batch 500 | Current Loss: 8.7080
Epoch [1/3] | Batch 550 | Current Loss: 8.3838
Epoch [1/3] | Batch 600 | Current Loss: 7.6927
Epoch [1/3] | Batch 650 | Current Loss: 7.8903
Epoch [1/3] | Batch 700 | Current Loss: 8.1739
Epoch [1/3] | Batch 750 | Current Loss: 7.9197
Epoch [1/3] | Batch 800 | Current Loss: 7.5296
Epoch [1/3] | Batch 850 | Current Loss: 7.7079
Epoch [1/3] | Batch 900 | Current Loss: 7.2957
Epoch [1/3] | Batch 950 | Current Loss: 8.5156
Epoch [1/3] | Batch 1000 | Current Loss: 7.9709
Epoch [1/3] | Batch 1050 | Current Loss: 8.2089
Epoch [1/3] | Batch 1100 | Current Loss: 7.9017
Epoch [1/3] | Batch 1150 | Current Loss: 7.4199
Epoch [1/3] | Batch 1200 | Current Loss: 7.2882
Epoch [1/3] | Batch 1250 | Current Loss: 7.9030
Epoch [1/3] | Batch 1300 | Current Loss: 7.4428
Epoch [1/3] | Batch 1350 | Current Loss: 7.3100
Epoch [1/3] | Batch 1400 | Current Loss: 7.0605
Epoch [1/3] | Batch 1450 | Current Loss: 7.5754
Epoch [1/3] | Batch 1500 | Current Loss: 7.6213
Epoch [1/3] | Batch 1550 | Current Loss: 6.9874
Epoch [1/3] | Batch 1600 | Current Loss: 7.5167
Epoch [1/3] | Batch 1650 | Current Loss: 8.0334
Epoch [1/3] | Batch 1700 | Current Loss: 7.2622
Epoch [1/3] | Batch 1750 | Current Loss: 6.7916
Epoch [1/3] | Batch 1800 | Current Loss: 7.6348
Epoch [1/3] | Batch 1850 | Current Loss: 7.6354
Epoch [1/3] | Batch 1900 | Current Loss: 7.6014
Epoch [1/3] | Batch 1950 | Current Loss: 6.6914
Epoch [1/3] | Batch 2000 | Current Loss: 7.4961
✅ Epoch 1 Completed | Average Loss: 7.8322

Epoch [2/3] | Batch 0 | Current Loss: 7.2671
Epoch [2/3] | Batch 50 | Current Loss: 8.4720
Epoch [2/3] | Batch 100 | Current Loss: 7.1473
Epoch [2/3] | Batch 150 | Current Loss: 6.7054
Epoch [2/3] | Batch 200 | Current Loss: 7.7389
Epoch [2/3] | Batch 250 | Current Loss: 6.7376
Epoch [2/3] | Batch 300 | Current Loss: 7.0406
Epoch [2/3] | Batch 350 | Current Loss: 7.1689
Epoch [2/3] | Batch 400 | Current Loss: 7.1575
Epoch [2/3] | Batch 450 | Current Loss: 6.9106
Epoch [2/3] | Batch 500 | Current Loss: 7.4011
Epoch [2/3] | Batch 550 | Current Loss: 7.8212
Epoch [2/3] | Batch 600 | Current Loss: 7.6219
Epoch [2/3] | Batch 650 | Current Loss: 6.7381
Epoch [2/3] | Batch 700 | Current Loss: 7.4194
```

```
Epoch [2/3] | Batch 750 | Current Loss: 7.0181
Epoch [2/3] | Batch 800 | Current Loss: 6.8207
Epoch [2/3] | Batch 850 | Current Loss: 6.4733
Epoch [2/3] | Batch 900 | Current Loss: 6.3920
Epoch [2/3] | Batch 950 | Current Loss: 6.9287
Epoch [2/3] | Batch 1000 | Current Loss: 7.1588
Epoch [2/3] | Batch 1050 | Current Loss: 6.9866
Epoch [2/3] | Batch 1100 | Current Loss: 6.8333
Epoch [2/3] | Batch 1150 | Current Loss: 6.7607
Epoch [2/3] | Batch 1200 | Current Loss: 7.3668
Epoch [2/3] | Batch 1250 | Current Loss: 7.6896
Epoch [2/3] | Batch 1300 | Current Loss: 6.7365
Epoch [2/3] | Batch 1350 | Current Loss: 6.9536
Epoch [2/3] | Batch 1400 | Current Loss: 7.1639
Epoch [2/3] | Batch 1450 | Current Loss: 6.8633
Epoch [2/3] | Batch 1500 | Current Loss: 7.8011
Epoch [2/3] | Batch 1550 | Current Loss: 5.8105
Epoch [2/3] | Batch 1600 | Current Loss: 6.5776
Epoch [2/3] | Batch 1650 | Current Loss: 6.9186
Epoch [2/3] | Batch 1700 | Current Loss: 7.1620
Epoch [2/3] | Batch 1750 | Current Loss: 6.7782
Epoch [2/3] | Batch 1800 | Current Loss: 6.4509
Epoch [2/3] | Batch 1850 | Current Loss: 6.5664
Epoch [2/3] | Batch 1900 | Current Loss: 6.9433
Epoch [2/3] | Batch 1950 | Current Loss: 6.5722
Epoch [2/3] | Batch 2000 | Current Loss: 6.8224
✅ Epoch 2 Completed | Average Loss: 6.9162

Epoch [3/3] | Batch 0 | Current Loss: 6.4924
Epoch [3/3] | Batch 50 | Current Loss: 6.5911
Epoch [3/3] | Batch 100 | Current Loss: 5.8419
Epoch [3/3] | Batch 150 | Current Loss: 6.4806
Epoch [3/3] | Batch 200 | Current Loss: 6.4677
Epoch [3/3] | Batch 250 | Current Loss: 5.8589
Epoch [3/3] | Batch 300 | Current Loss: 6.6455
Epoch [3/3] | Batch 350 | Current Loss: 6.6749
Epoch [3/3] | Batch 400 | Current Loss: 7.6032
Epoch [3/3] | Batch 450 | Current Loss: 7.0004
Epoch [3/3] | Batch 500 | Current Loss: 5.1841
Epoch [3/3] | Batch 550 | Current Loss: 6.0800
Epoch [3/3] | Batch 600 | Current Loss: 6.2975
Epoch [3/3] | Batch 650 | Current Loss: 6.3560
Epoch [3/3] | Batch 700 | Current Loss: 6.6618
Epoch [3/3] | Batch 750 | Current Loss: 5.7452
Epoch [3/3] | Batch 800 | Current Loss: 6.7288
Epoch [3/3] | Batch 850 | Current Loss: 6.4137
Epoch [3/3] | Batch 900 | Current Loss: 6.5591
Epoch [3/3] | Batch 950 | Current Loss: 6.2783
Epoch [3/3] | Batch 1000 | Current Loss: 6.4855
Epoch [3/3] | Batch 1050 | Current Loss: 6.5478
Epoch [3/3] | Batch 1100 | Current Loss: 6.0470
Epoch [3/3] | Batch 1150 | Current Loss: 6.3272
Epoch [3/3] | Batch 1200 | Current Loss: 7.0819
Epoch [3/3] | Batch 1250 | Current Loss: 6.5213
Epoch [3/3] | Batch 1300 | Current Loss: 5.7327
Epoch [3/3] | Batch 1350 | Current Loss: 5.5333
Epoch [3/3] | Batch 1400 | Current Loss: 6.3789
Epoch [3/3] | Batch 1450 | Current Loss: 7.1007
Epoch [3/3] | Batch 1500 | Current Loss: 6.8391
Epoch [3/3] | Batch 1550 | Current Loss: 6.6946
```

```
Epoch [3/3] | Batch 1600 | Current Loss: 7.0003
Epoch [3/3] | Batch 1650 | Current Loss: 6.8068
Epoch [3/3] | Batch 1700 | Current Loss: 6.7169
Epoch [3/3] | Batch 1750 | Current Loss: 6.3911
Epoch [3/3] | Batch 1800 | Current Loss: 6.5669
Epoch [3/3] | Batch 1850 | Current Loss: 6.9241
Epoch [3/3] | Batch 1900 | Current Loss: 5.6067
Epoch [3/3] | Batch 1950 | Current Loss: 5.5382
Epoch [3/3] | Batch 2000 | Current Loss: 6.4687
✅ Epoch 3 Completed | Average Loss: 6.4087
```

In [ ]:

```
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import torch.nn.functional as F


test_loader = DataLoader(test_data, batch_size=32, shuffle=False, num_wor

model.eval()
outliers = []

print("🕵️ Hunting for outliers in the test set (Confident but 'Wrong' pai

with torch.no_grad():
    for images, style_targets, artist_targets in test_loader:
        images = images.to(device)
        style_targets = style_targets.to(device)


        style_preds, artist_preds = model(images)

        probs = F.softmax(style_preds, dim=1)
        max_probs, predicted_styles = torch.max(probs, dim=1)


        for i in range(len(images)):
            # If the model is >85% sure, but the label says otherwise
            if predicted_styles[i] != style_targets[i] and max_probs[i] >

                # Decode the numbers back into text (e.g., "Impressionism
                pred_text = style_enc.inverse_transform([predicted_styles
                actual_text = style_enc.inverse_transform([style_targets[

                outliers.append({
                    'image': images[i].cpu(),
                    'predicted': pred_text,
                    'actual': actual_text,
                    'confidence': max_probs[i].item() * 100
                })


        if len(outliers) >= 4:
            break

print(f"✅ Found {len(outliers)} fascinating outliers!")


fig, axes = plt.subplots(1, len(outliers), figsize=(20, 5))
for idx, outlier in enumerate(outliers):
```

```
    img = outlier['image']
    img = img * torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)
    img = img + torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    img = img.clamp(0, 1).permute(1, 2, 0).numpy()

    axes[idx].imshow(img)
    axes[idx].axis('off')
    axes[idx].set_title(f"Label: {outlier['actual']}\nModel Guessed: {out

plt.tight_layout()
plt.show()
```

```
🕵️ Hunting for outliers in the test set (Confident but 'Wrong' painting
s)...
✅ Found 4 fascinating outliers!
```